

GUÍA DE ESTUDIO - Práctico N°5

De Texto Caótico a Dataset Limpio

INTRODUCCIÓN

Esta guía te llevará paso a paso para resolver el práctico. Cada sección incluye: - **Concepto teórico** que necesitas entender - **Ejercicio práctico** para aplicar el concepto - **Código de ejemplo** para guiarte - **Checkpoint** para verificar que vas bien

PARTE 0: PREPARACIÓN DEL ENTORNO

Paso 0.1: Crear la estructura de carpetas

Antes de escribir código, necesitas organizar tu proyecto:

```
proyecto_final/
├── main.py           # Tu código principal
├── data/
│   ├── raw/
│   │   └── ingesta_cruda.txt # Archivo de entrada (se te proporciona)
│   └── processed/         # Aquí se guardarán los archivos limpios
└── logs/
    └── sistema.log        # Se creará automáticamente
```

EJERCICIO: Crea estas carpetas en tu computadora y coloca el archivo `ingesta_cruda.txt` en `data/raw/`.

PARTE 1: INGESTA ROBUSTA

Concepto Clave: Lectura de Archivos con Manejo de Errores

Cuando leemos archivos, pueden ocurrir errores: - El archivo no existe (`FileNotFoundError`) - Problemas de codificación (`UnicodeDecodeError`)

Paso 1.1: Importar las librerías necesarias

```
import re          # Para expresiones regulares
import logging     # Para registrar eventos
import csv         # Para guardar en CSV
import json        # Para guardar en JSON
import os          # Para manejar rutas de archivos
```

EJERCICIO: Crea un archivo `main.py` e importa estas librerías.

Paso 1.2: Configurar el sistema de Logging

El logging nos permite registrar qué pasó durante la ejecución:

```
# Configurar logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("logs/sistema.log", encoding='utf-8'),
        logging.StreamHandler() # También muestra en consola
    ]
)
logger = logging.getLogger(__name__)
```

PREGUNTA DE AUTOEVALUACIÓN: ¿Para qué sirve cada nivel de logging (INFO, WARNING, ERROR)? - `INFO` : Información general sobre el flujo del programa - `WARNING` : Algo inesperado pero no crítico (ej: dato duplicado) - `ERROR` : Un error que impide completar una operación

Paso 1.3: Crear la función de lectura

```
def cargar_archivo(ruta_archivo):
    """Lee el archivo y devuelve una lista de líneas."""
    logger.info(f"Intentando leer: {ruta_archivo}")

    try:
        with open(ruta_archivo, 'r', encoding='utf-8') as archivo:
            lineas = archivo.readlines()
            logger.info(f"Archivo cargado: {len(lineas)} líneas")
            return lineas

    except FileNotFoundError:
        logger.error("El archivo no existe")
        return None

    except UnicodeDecodeError:
        logger.error("Error de codificación")
        # Intento con otra codificación
        try:
            with open(ruta_archivo, 'r', encoding='latin-1') as archivo:
                return archivo.readlines()
        except:
            return None
```

CHECKPOINT 1: Prueba tu función con:

```
lineas = cargar_archivo("data/raw/ingesta_cruda.txt")
if lineas:
    print(f"✓ Se leyeron {len(lineas)} líneas")
```

PARTE 2: MOTOR DE LIMPIEZA - REGEX

Concepto Clave: Expresiones Regulares

Las expresiones regulares (Regex) permiten buscar patrones en texto.

Símbolos básicos que usarás:

Símbolo	Significado	Ejemplo
<code>\d</code>	Un dígito (0-9)	<code>\d+</code> = uno o más dígitos
<code>\s</code>	Espacio en blanco	<code>\s*</code> = cero o más espacios
<code>+</code>	Uno o más	<code>\d+</code> = "123", "7", "99999"
<code>*</code>	Cero o más	<code>\s*</code> = "", " ", " "
<code>[]</code>	Conjunto de caracteres	<code>[a-zA-Z]</code> = cualquier letra
<code>^</code>	Inicio de línea	<code>^###</code> = línea que empieza con ###
<code>\$</code>	Fin de línea	<code>\.com\$</code> = termina en .com

Paso 2.1: Detectar líneas de sistema (que debemos ignorar)

Observa el archivo `ingesta_cruda.txt`. Hay líneas que NO son clientes: - `###`

INICIO DE LOTE - SISTEMA DE INGESTA V2.4 `###` -

----- - Registro Corrupto `####`

Error 404 `####`

```
def es_linea_sistema(linea):
    """Detecta líneas que deben ignorarse."""
    linea_limpia = linea.strip()

    # Líneas vacías
    if not linea_limpia:
        return True

    # Encabezados (empiezan o terminan con ###)
    if linea_limpia.startswith('###') or linea_limpia.endswith('###'):
```

```

        return True

    # Separadores (solo guiones)
    if re.match(r'^-{5,}$', linea_limpia):
        return True

    # Errores del sistema
    if 'Error' in linea_limpia and '####' in linea_limpia:
        return True

    return False

```

EJERCICIO: Prueba esta función con diferentes líneas:

```

print(es_linea_sistema("### INICIO ###"))      # True
print(es_linea_sistema("Juan Perez | 123"))    # False
print(es_linea_sistema("-----"))            # True

```

Paso 2.2: Extraer el NOMBRE del cliente

El nombre puede venir en diferentes formatos: - Cliente: Ana Lopez - Tel: ... -
 Cli: Maria Garcia - Tel: ... - Juan Perez | 351 555 1234 | ...

Estrategia: Primero limpiar prefijos, luego extraer el nombre.

```

def extraer_nombre(linea):
    """Extrae el nombre del cliente."""
    # Lista de prefijos a eliminar
    prefijos = [
        r'^ID:\s*\d+\s*-\s*',      # ID: 001 -
        r'^Cliente Nuevo:\s*',    # Cliente Nuevo:
        r'^Cliente:\s*',          # Cliente:
        r'^Contacto:\s*',        # Contacto:
        r'^Socio:\s*',            # Socio:
        r'^Cli:\s*',              # Cli:
    ]

    linea_limpia = linea.strip()

    # Eliminar cada prefijo
    for prefijo in prefijos:
        linea_limpia = re.sub(prefijo, '', linea_limpia, flags=re.IGNORECASE)

    # Ahora separamos por el delimitador (puede ser |, -, ;, /)
    # Tomamos la primera parte que debería ser el nombre
    partes = re.split(r'\s*[-|/]\s*(?:Tel|Cel|Mail|\d{3})', linea_limpia, maxsplit=1)

    if partes and partes[0].strip():

```

```
nombre = partes[0].strip()
return nombre.title() # Convierte a Title Case

return None
```

PREGUNTA: ¿Qué hace `.title()` ? - Convierte "PEDRO ALVAREZ" → "Pedro Alvarez"
- Convierte "ana lopez" → "Ana Lopez"

Paso 2.3: Extraer el TELÉFONO

El teléfono debe tener exactamente 10 dígitos. Puede venir como: - 351-456-7890 -
(351) 555 1234 - 3514567890

Estrategia: Extraer TODOS los dígitos, luego validar que sean 10.

```
def extraer_telefono(linea):
    """Extrae y normaliza el teléfono a 10 dígitos."""
    # Extraer todos los dígitos
    digitos = re.findall(r'\d', linea)
    telefono = ''.join(digitos)

    # Validar exactamente 10 dígitos
    if len(telefono) == 10:
        return telefono

    # Si hay más de 10, tomamos los últimos 10
    if len(telefono) > 10:
        return telefono[-10:]

    # Si hay menos de 10, es inválido
    return None
```

EJERCICIO: Prueba con estas líneas:

```
print(extraer_telefono("Tel: 351-456-7890")) # "3514567890"
print(extraer_telefono("(351) 555 1234"))    # "3515551234"
print(extraer_telefono("Tel: 123"))          # None (muy corto)
```

Paso 2.4: Extraer el EMAIL

El email debe tener formato: `usuario@dominio.extension`

```
def extraer_email(linea):
    """Extrae y valida el email."""
    # Patrón: letras/números/puntos @ dominio . extensión(2+ letras)
    patron_email = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
```

```
match = re.search(patron_email, linea)

if match:
    return match.group().lower() # Convertir a minúsculas

return None
```

DESGLOSE DEL PATRÓN:

```
[a-zA-Z0-9._%+-]+ → usuario (uno o más caracteres válidos)
@                → arroba literal
[a-zA-Z0-9.-]+   → dominio (uno o más caracteres)
\.              → punto literal (escapado con \)
[a-zA-Z]{2,}     → extensión (al menos 2 letras: com, ar, org)
```

EJERCICIO: ¿Por qué `robertog@gmail` es INVÁLIDO? Respuesta: Porque no tiene extensión después del punto (falta `.com`)

PARTE 3: GESTIÓN EN MEMORIA

Concepto Clave: Lista de Diccionarios

Usamos una lista de diccionarios porque: - Es más legible: `cliente["email"]` vs `cliente[2]` - Es más flexible: podemos agregar campos fácilmente - Se convierte directamente a JSON

Paso 3.1: Estructura de datos global

```
# Variables globales
clientes_limpios = []          # Lista de diccionarios
emails_registrados = set()    # Para detectar duplicados
estadisticas = {
    "total_lineas": 0,
    "registros_validos": 0,
    "registros_descartados": 0,
    "emails_gmail": 0
}
```

Paso 3.2: Función para procesar cada línea

```
def procesar_linea(linea, numero_linea):
    """Procesa una línea y devuelve un diccionario o None."""
    global emails_registrados, estadisticas

    # 1. Ignorar líneas de sistema
    if es_linea_sistema(linea):
        logger.info(f"Línea {numero_linea} ignorada: línea de sistema")
        estadisticas["registros_descartados"] += 1
        return None

    # 2. Extraer datos
    nombre = extraer_nombre(linea)
    telefono = extraer_telefono(linea)
    email = extraer_email(linea)

    # 3. Validar: debe tener nombre
    if not nombre:
        logger.warning(f"Línea {numero_linea} descartada: sin nombre")
        estadisticas["registros_descartados"] += 1
        return None
```

```

# 4. Validar: debe tener email
if not email:
    logger.warning(f"Línea {numero_linea} descartada: email inválido - {nombre}")
    estadisticas["registros_descartados"] += 1
    return None

# 5. Verificar duplicados
if email in emails_registrados:
    logger.warning(f"Línea {numero_linea} descartada: email duplicado ({email})")
    estadisticas["registros_descartados"] += 1
    return None

# 6. Registrar email
emails_registrados.add(email)

# 7. Teléfono faltante = "Desconocido"
if not telefono:
    telefono = "Desconocido"
    logger.info(f"Línea {numero_linea}: teléfono marcado como Desconocido")

# 8. Contar emails de Gmail
if email.endswith('@gmail.com'):
    estadisticas["emails_gmail"] += 1

estadisticas["registros_validos"] += 1

# 9. Retornar el diccionario limpio
return {
    "nombre": nombre,
    "telefono": telefono,
    "email": email
}

```

Paso 3.3: Función principal de carga y limpieza

```

def cargar_y_limpiar():
    """Carga el archivo y procesa todas las líneas."""
    global clientes_limpios, emails_registrados, estadisticas

    # Reiniciar datos
    clientes_limpios = []
    emails_registrados = set()

    lineas = cargar_archivo("data/raw/ingesta_cruda.txt")

    if lineas is None:
        print("❌ No se pudo cargar el archivo")
        return False

    estadisticas["total_lineas"] = len(lineas)

```

```
for i, linea in enumerate(lineas, start=1):
    cliente = procesar_linea(linea, i)
    if cliente:
        clientes_limpios.append(cliente)

print(f"✅ Proceso completado:")
print(f"    - Líneas leídas: {estadisticas['total_lineas']}")
print(f"    - Registros válidos: {estadisticas['registros_validos']}")
print(f"    - Registros descartados: {estadisticas['registros_descartados']}")

return True
```

CHECKPOINT 2: Después de ejecutar `cargar_y_limpiar()` , deberías tener: - 19 registros válidos - 9 registros descartados

PARTE 4: MENÚ INTERACTIVO

Paso 4.1: Función de búsqueda

```
def buscar_cliente():
    """Busca un cliente por nombre."""
    if not clientes_limpios:
        print("⚠ No hay clientes cargados")
        return

    nombre_buscar = input("Ingresa el nombre a buscar: ").strip().lower()

    encontrados = [c for c in clientes_limpios
                    if nombre_buscar in c["nombre"].lower()]

    if encontrados:
        print(f"\n🔍 Se encontraron {len(encontrados)} resultado(s):")
        for c in encontrados:
            print(f"    Nombre: {c['nombre']}")
            print(f"    Teléfono: {c['telefono']}")
            print(f"    Email: {c['email']}")
            print("    " + "-" * 30)
    else:
        print(f"❌ No se encontró '{nombre_buscar}'")
```

Paso 4.2: Función de estadísticas

```
def mostrar_estadisticas():
    """Muestra estadísticas del proceso."""
    if estadisticas["total_lineas"] == 0:
        print("⚠ No hay datos cargados")
        return

    validos = estadisticas["registros_validos"]
    gmail = estadisticas["emails_gmail"]
    porcentaje_gmail = (gmail / validos * 100) if validos > 0 else 0

    print("\n📊 ESTADÍSTICAS:")
    print(f"    Total líneas leídas:      {estadisticas['total_lineas']}")
    print(f"    Registros válidos:        {validos}")
    print(f"    Registros descartados:    {estadisticas['registros_descartados']}")
    print(f"    Emails de Gmail:          {gmail} ({porcentaje_gmail:.1f}%)")
```

Paso 4.3: Función de eliminación

```
def eliminar_cliente():
    """Elimina un cliente por email."""
    global clientes_limpios

    if not clientes_limpios:
        print("⚠ No hay clientes cargados")
        return

    email_eliminar = input("Email del cliente a eliminar: ").strip().lower()

    for i, cliente in enumerate(clientes_limpios):
        if cliente["email"] == email_eliminar:
            eliminado = clientes_limpios.pop(i)
            print(f"✅ Cliente eliminado: {eliminado['nombre']}")
            logger.info(f"Cliente eliminado: {eliminado['nombre']}")
            return

    print(f"❌ No se encontró cliente con email '{email_eliminar}'")
```

Paso 4.4: Mostrar todos los clientes

```
def mostrar_clientes():
    """Muestra todos los clientes."""
    if not clientes_limpios:
        print("⚠ No hay clientes cargados")
        return

    print(f"\n📋 LISTADO DE CLIENTES ({len(clientes_limpios)} registros):")
    print("=" * 70)
    for i, c in enumerate(clientes_limpios, start=1):
        print(f"{i:2}. {c['nombre']:<25} | {c['telefono']:<12} | {c['email']}>")
    print("=" * 70)
```

PARTE 5: PERSISTENCIA

Paso 5.1: Guardar en CSV

```
def guardar_csv():
    """Exporta los datos a CSV."""
    try:
        with open("data/processed/clientes_limpios.csv", 'w',
                  newline='', encoding='utf-8') as archivo:
            campos = ["nombre", "telefono", "email"]
            writer = csv.DictWriter(archivo, fieldnames=campos)
            writer.writeheader()
            writer.writerows(clientes_limpios)
            logger.info("Datos exportados a CSV")
            return True
    except Exception as e:
        logger.error(f"Error al guardar CSV: {e}")
        return False
```

Paso 5.2: Guardar en JSON

```
def guardar_json():
    """Exporta los datos a JSON."""
    try:
        with open("data/processed/clientes_limpios.json", 'w',
                  encoding='utf-8') as archivo:
            json.dump(clientes_limpios, archivo, indent=2, ensure_ascii=False)
            logger.info("Datos exportados a JSON")
            return True
    except Exception as e:
        logger.error(f"Error al guardar JSON: {e}")
        return False
```

Paso 5.3: Función guardar y salir

```
def guardar_y_salir():
    """Guarda los datos y termina el programa."""
    if not clientes_limpios:
        print("⚠ No hay datos para guardar")
        return False

    print("\n💾 Guardando datos...")
```

```
exito_csv = guardar_csv()
exito_json = guardar_json()

if exito_csv and exito_json:
    print("✅ Archivos guardados exitosamente")
    return True
else:
    print("⚠️ Hubo errores al guardar")
    return False
```

PARTE 6: MENÚ PRINCIPAL

```
def mostrar_menu():
    print("\n" + "=" * 50)
    print("    SISTEMA DE GESTIÓN DE CLIENTES")
    print("=" * 50)
    print("    1. Cargar y Limpiar datos")
    print("    2. Buscar cliente")
    print("    3. Ver estadísticas")
    print("    4. Eliminar cliente")
    print("    5. Mostrar todos los clientes")
    print("    6. Guardar y Salir")
    print("=" * 50)

def main():
    logger.info("INICIO DEL PROGRAMA")
    print("\n🚀 Bienvenido al Sistema de Gestión de Clientes")

    while True:
        mostrar_menu()
        opcion = input("\nSelecciona una opción (1-6): ").strip()

        if opcion == "1":
            cargar_y_limpiar()
        elif opcion == "2":
            buscar_cliente()
        elif opcion == "3":
            mostrar_estadisticas()
        elif opcion == "4":
            eliminar_cliente()
        elif opcion == "5":
            mostrar_clientes()
        elif opcion == "6":
            if guardar_y_salir():
                print("\n¡Hasta luego! 🙌")
                break
        else:
            print("\n❌ Opción no válida")

if __name__ == "__main__":
    main()
```

CHECKLIST FINAL

Antes de entregar, verifica:

- ☐ El programa lee el archivo sin errores
 - ☐ Se genera el archivo `sistema.log` con los eventos
 - ☐ Se procesan 19 clientes válidos
 - ☐ Los emails duplicados se descartan (conservando el primero)
 - ☐ Los teléfonos faltantes se marcan como "Desconocido"
 - ☐ Se genera `clientes_limpios.csv`
 - ☐ Se genera `clientes_limpios.json`
 - ☐ El menú funciona correctamente
-

PREGUNTAS PARA EL INFORME DE REFLEXIÓN

1. **¿Por qué lista de diccionarios?**

- 2. Acceso por nombre de campo (`cliente["email"]`) vs índice (`cliente[2]`)
- 3. Más legible y menos propenso a errores

4. Se convierte fácilmente a JSON

5. **¿Cómo funciona el patrón de teléfono?**

6. `re.findall(r'\d', linea)` extrae todos los dígitos

7. Luego validamos que sean exactamente 10

8. **¿Qué estrategia usaste para datos faltantes?**

- 9. Email faltante/inválido: DESCARTE (es el identificador único)
- 10. Teléfono faltante: MARCACIÓN como "Desconocido"

11. Líneas de sistema: IGNORADAS

12. **¿Cómo escalaría a 1 millón de filas?**

- 13. Lectura por chunks (no cargar todo en memoria)
- 14. Formato Parquet para persistencia eficiente
- 15. Procesamiento con Pandas/Dask